

Edge-to-Edge System UI Configuration in Flutter

This document explains how to configure **edge-to-edge system UI** in Flutter using `SystemChrome`. This setup is commonly used for modern UI designs where content is allowed to draw behind the status bar and system navigation bar.

Overview

The following configuration: - Enables edge-to-edge layout - Makes the status bar and navigation bar transparent - Controls icon brightness for visibility

This is typically placed inside the `initState()` method of a `StatefulWidget`.

Code Explained

1. Enable Edge-to-Edge Mode

```
SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
```

What it does: - Allows your app UI to render behind system bars - Required for immersive and modern Android UI (Android 10+) - Works well with custom navigation bars and animations

2. Configure System Bar Appearance

```
SystemChrome.setSystemUIOverlayStyle(  
  const SystemUiOverlayStyle(  
    statusBarColor: Colors.transparent,  
    statusBarIconBrightness: Brightness.dark,  
    systemNavigationBarColor: Colors.transparent,  
    systemNavigationBarIconBrightness: Brightness.light,  
  ),  
);
```

Status Bar

- **Transparent background** allows content behind it
- **Dark icons** ensure visibility on light backgrounds

Navigation Bar (Android)

- **Transparent navigation bar** for floating UI designs
 - **Light icons** for contrast on dark backgrounds
-

Important Considerations

1. SafeArea Usage (Required)

Because content renders behind system bars, always protect critical UI:

```
SafeArea(  
  child: YourWidget(),  
)
```

Or apply manual padding:

```
EdgeInsets.only(  
  bottom: MediaQuery.of(context).padding.bottom,  
)
```

2. Restore System UI on Dispose (Best Practice)

If this configuration is screen-specific, reset system UI when leaving the screen:

```
@override  
void dispose() {  
  SystemChrome.setEnabledSystemUIMode(  
    SystemUiMode.manual,  
    overlays: SystemUiOverlay.values,  
  );  
  
  SystemChrome.setSystemUIOverlayStyle(  
    const SystemUiOverlayStyle(  
      statusBarColor: Colors.black,  
      statusBarIconBrightness: Brightness.light,  
      systemNavigationBarColor: Colors.black,  
      systemNavigationBarIconBrightness: Brightness.light,  
    ),  
  );  
}
```

```
    super.dispose();  
}
```

Recommended Use Cases

- Custom bottom navigation bars
 - Glassmorphism UI designs
 - Full-screen animations and wave effects
 - Apps using gesture navigation
-

Notes

- On **iOS**, navigation bar color changes are ignored (handled by Cupertino system)
 - Always test on devices with **gesture navigation enabled**
 - Combine with `extendBody: true` for bottom navigation layouts
-

Summary

Using `SystemUiMode.edgeToEdge` with transparent system bars gives you full control over modern UI layouts. Always pair it with `SafeArea` or padding to avoid UI overlap and reset the system UI when the screen is disposed.

✓ This setup is ideal for advanced UI components such as custom navigation bars, animated backgrounds, and immersive layouts.